

Schedule

- Hands-on
- pip install pre-built wheel, then link custom dialect (that participants write during exercises). same infrastructure shared for all sessions.
- experienced + beginner audience

Theme ideas:

- Scalable vectors
 - Transform dialect
 - Interfaces
 - CUDA TileIR
 - Can rent GPU to run on
 - mem2reg
 - Dataflow analysis
 - forwards/backwards
 - Bounds analysis
 - Bufferization
 - Rewrites
 - Symbols
 - PDL
 - Eqsat in MLIR
 - Python bindings
 - Assembly dialects
 - SMT dialect
 - Lowering to LLVM
 - Deep dive in IR data structure
 - Deep dive in TblGen
-
- Structured Control Flow
 - Parser/Printer (Can be part of ODS)

Unifying problem ideas:

- Digital signal processing (the modeling of f32 over time)

Links:

- <https://github.com/DavidGinten/ML-compiler-exercise/tree/main>

Schedule: First track

1. Compiler basics: (SADAY, TOBIAS)
 - a. SSA IR, etc.
 - b. What is a pass
 - c.
2. Introduction of running example / topic. What IR are we going to use throughout the remaining sessions? TODO: Topic? DSP? Complex numbers? Something else? (THEO)
 - a. Introduction to MLIR.
 - b. Operations / IR Structure / Regions
 - c. How to represent (2) in IR. How do you design IR in MLIR?
3. Pass infrastructure: implement your first MLIR pass. (MATTHIAS)
4. Pattern infrastructure (incl. PatternRewriter, Builder API) (MATTHIAS)
5. Interfaces (MATTHIAS)
6. Core transformations: Canonicalization / folding / CSE / TrivialDCE pass / etc (MARKUS)
7. Data flow analysis (MATHIEU)
8. Mem2Reg (THEO)
9. Deep dive in TblGen + ODS (MARKUS)
10. Deep dive in IR datastructure / performance (MATHIEU)
11. GPU CUDA Tile IR (THEO)

Review Lecture Status: July 19

Introduction

Introduction Session

Lead: Kunwar & Tobias

Compiler Introduction

Compiler Introduction

Lead: Tobias & Saday

MLIR IR Introduction

MLIR IR Introduction

Lead: Theo

High-level Plan/Goals:

- No C++ in this session.
- Very interactive, basically only exercises

Plan:

Comments:

- Can we have structured control flow shown in the IR?

Poster Session

Poster Session

Owner: ?

Building Transformations

Building Transformations

Lead: Matthias

Session 5: Pass Infrastructure + C++ API for modifying IR

Duration: 90 min

Theme: How do I run code over the IR, and how do I change the IR by hand?

Learning objectives: By the end, students can: write and register a pass from TableGen, run it standalone/nested via mlir-opt, navigate the SSA + region IR model in C++, and mutate IR directly with builders and Operation/Value/Block APIs.

Part A — Pass Infrastructure (≈50 min)

1. Motivation & the conceptual model (8 min)

What a pass is: a transformation scoped to one operation instance ("the current op").

A pass manager is a pipeline anchored at an op type; nesting mirrors the op hierarchy.

Big mental model slide: PassManager → OpPassManager → Pass, with OpToOpPassAdaptor driving nesting.

2. Anatomy of a pass (12 min)

Pass, OperationPass<OpT>, OperationPass<>, InterfacePass<I>, PassWrapper (mlir/include/mlir/Pass/Pass.h).

The three things you implement: runOnOperation(), getOperation(), signalPassFailure().

Live walkthrough of a real pass: `mlir/lib/Transforms/CSE.cpp` (small, readable, uses analyses + statistics).

3. Declaring passes with TableGen (12 min)

`def Pass<"arg", "anchor-op"> in PassBase.td;` the anchor op argument and what "" (generic) means.

The generated Passes.h.inc 3-include pattern: `GEN_PASS_DECL`, `GEN_PASS_DEF_*`, `GEN_PASS_REGISTRATION`.

Real example: `mlir/include/mlir/Dialect/Arith/Transforms/Passes.td` + `EmulateWideInt.cpp` (struct `Pass : impl::Base<Pass>`, using `Base::Base`).
options (`Option<>`, `ListOption<>`), statistics (`Statistic<>`), `dependentDialects`.

4. Pass managers, nesting & pipelines (10 min)

`addPass`, `nest<func::FuncOp>()`, `addNestedPass<>`; implicit vs explicit nesting.

Textual pipeline syntax students must memorize: `builtin.module(cse, func.func(canonicalize,cse))`, options as `pass{opt=val}`.

Registration paths: `PassRegistration<>`, `PassPipelineRegistration<>`, dialect bundles (`registerArithPasses()`), and `registerAllPasses()`.

5. Analyses, threading & debugging (8 min)

getAnalysis<T>(), getCachedAnalysis<T>(), and the preservation contract:

markAllAnalysesPreserved() / markAnalysesPreserved<DominanceInfo>() (show in CSE).

Threading model: sibling isolated-from-above ops run in parallel $\Rightarrow$ the "only touch the current op and its descendants" rule, and why it exists.

Debugging toolbox: --mlir-print-ir-after-all, --mlir-pass-statistics, --mlir-pass-timing, crash reproducer.

Part B — C++ API for Modifying IR ($\approx$ 35 min)

6. The IR data model in C++ (8 min)

Operation $\rightarrow$ Region $\rightarrow$ Block $\rightarrow$ Operation; Value = OpResult | BlockArgument; uses tracked via OpOperand.

Use-def chains: getUsers(), getUses(), use_empty(), getDefiningOp() (null for block args).

7. Builders & creating IR (12 min)

Builder (context-global: types/attrs, e.g. getI32Type(), getIntegerAttr()) vs OpBuilder (insertion-point aware).

Insertion points + OpBuilder::InsertionGuard (RAII), setInsertionPointToStart/After, atBlockBegin/End.

Canonical op creation: OpTy::create(builder, loc, ...) — note the old builder.create<OpTy>(…) is now deprecated. ImplicitLocOpBuilder for implicit locations.

Creating blocks/regions: createBlock, block arguments, Block::splitBlock.

8. Direct mutation (12 min)

Operation: setOperand, eraseOperand, moveBefore/After, setAttr, erase() (requires no uses).

Value::replaceAllUsesWith, Operation::replaceAllUsesWith, replaceUsesOfWith.

Safe iteration while mutating: llvm::make_early_inc_range(value.getUses()).

Walking IR: op->walk(...), WalkResult (advance/skip/interrupt), post-order erase safety.

9. Bridge to Session 6 (3 min)

Direct mutation is fine in passes/frontends, but it bypasses listeners. Inside pattern-based transforms you must use a rewriter — segue to Session 6.

Hands-on / demo ideas: write a func.func pass that inserts a constant + an arith.addi, then rewrites all uses of an argument; run it standalone and nested; observe analysis invalidation via statistics.

Session 6: Rewriter, Pattern Driver Infrastructure

Duration: 90 min

Theme: The idiomatic way to transform IR — declarative patterns applied by a driver.

Learning objectives: Students can: write OpRewritePatterns with correct rewriter discipline, assemble and freeze a RewritePatternSet, drive them with both the greedy and walk drivers,

understand how folding integrates, and know where dialect conversion fits (and why it's a separate framework).

0. Recap & framing (5 min)

Why patterns over hand-written walks: local, composable, benefit-ordered, applied automatically.

The pipeline slide: RewritePatternSet → FrozenRewritePatternSet → PatternApplicator → Driver → PatternRewriter.

1. The rewriter hierarchy (12 min)

OpBuilder → RewriterBase → {IRRewriter, PatternRewriter} (all in PatternMatch.h).

Why you MUST use the rewriter inside patterns: drivers act as a RewriterBase::Listener; direct mutation breaks worklist bookkeeping (debug builds fatal-error on it).

Key RewriterBase API: replaceOp, replaceOpWithNewOp<OpTy>, eraseOp, modifyOpInPlace (start/finalize/cancelOpModification), inlineRegionBefore, mergeBlocks, notifyMatchFailure.

IRRewriter for standalone (non-pattern) transforms that still need notifications.

2. Writing patterns (15 min)

Pattern → RewritePattern → OpRewritePattern<OpTy>; also OpInterfaceRewritePattern, OpTraitRewritePattern.

Just one method: matchAndRewrite(OpTy, PatternRewriter&) (no split match/rewrite in C++ base).

PatternBenefit (0–65535) and how ordering/ties work.

Live read of a real, tiny pattern: SelectToExtUI in mlir/lib/Dialect/Arith/IR/ArithOps.cpp (matchPattern, m_One(), replaceOpWithNewOp).

hasBoundedRewriteRecursion, getGeneratedOps (debugging recursion/cost).

3. Assembling & freezing patterns (8 min)

RewritePatternSet::add<...>(ctx); the populateXxxPatterns(RewritePatternSet&) convention.

FrozenRewritePatternSet: indexing by op name, expanding interface/trait patterns, shareable across threads, enable/disable labels.

4. The greedy driver + folding (20 min)

Entry points: applyPatternsGreedily(Region&/Operation*),

applyOpPatternsGreedily(ArrayRef<Operation*>).

GreedyRewriteConfig knobs: useTopDownTraversal, maxIterations (default 10), maxNumRewrites, scope, strictness, enableFolding (default on), enableConstantCSE, regionSimplificationLevel, listener.

Worklist semantics: LIFO + O(1) removal; per op → dead-check → fold → pattern match; listener re-enqueues inserted/modified ops and operand producers on erase.

Folding inside the loop: Operation::fold() / hasFolder, OpFoldResult, materializeConstant; folding runs before patterns, plus constant CSE on worklist population.

Convergence and what "didn't converge" diagnostics mean.

5. The walk pattern driver (10 min)

walkAndApplyPatterns (mlir/lib/Transforms/Utils/WalkPatternRewriteDriver.cpp) — a simpler alternative driver.

Single deterministic walk, no worklist, no fixpoint iteration, no folding: each op is visited once.

When to prefer it: you know one pass is enough (e.g. straightforward lowering/expansion), you want predictable cost and ordering, and you don't need re-visitation. Still uses PatternRewriter discipline + listener for safe erase/insert during the walk.

Contrast slide: greedy (fixpoint, folding, more expensive, can loop) vs walk (one shot, cheap, no convergence guarantees).

6. Declarative & advanced patterns (10 min)

DRR (Pat<...> in TableGen) → compiled to C++ matchAndRewrite — show a Toy

ToyCombine.td snippet.

PDL/PDLL as the declarative pattern-as-data path (match/rewrite split at bytecode level) — mention, don't deep-dive.

Transform dialect: transform.apply_patterns wraps the greedy driver.

7. Dialect conversion — brief positioning (7 min)

A separate framework for type/dialect legalization, not a same-type simplifier.

Vocabulary only: ConversionPattern/OpConversionPattern (note the adapted operands), ConversionPatternRewriter, TypeConverter, ConversionTarget (legal/illegal/dynamic), applyPartialConversion/applyFullConversion.

Contrast table (greedy/walk vs conversion) and the key warning: don't drop conversion patterns into a greedy pass — they need the conversion driver and rewriter. Full treatment is its own session.

Core Transformations

Core Transformations

Lead: Markus, Max

Canonicalizations, CSE, DCE, folding

ODS

ODS

Lead: Markus

Overall Goals:

- Defining dialects, C++ APIs

Interfaces

Interfaces

Owner: Mathias

Mem2Reg

Mem2Reg

Owner: Théo

Data Flow Analysis

Data Flow Analysis

Owner: Mathieu

IR Data Structures and Performance

IR Data Structures and Performance

Owner: Mathieu

CUDA Tile IR

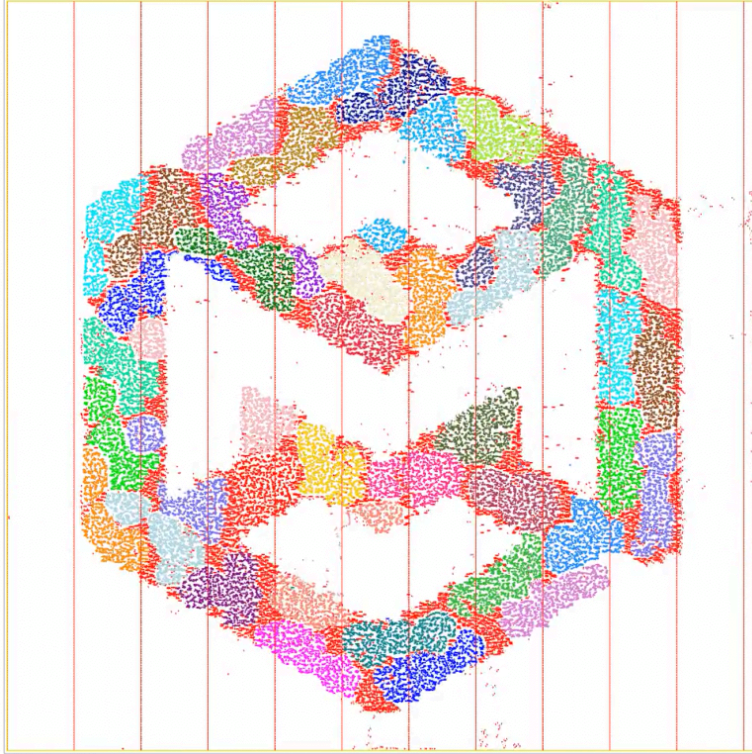
CUDA Tile IR

Owner: ?

CIRCT

CIRCT

Owner: Louis



High Level view of this session

- This track is about a use-case of MLIR
- Is hardware design possible with CIRCT ?
- Compilation -> Generation in the hardware world: create the machine and not use the machine but with the same techniques.

A plan of this session

- 1. Hardware design basic
 - The need for specialization
 - Verilog / VHDL are the ir

- There is combinational and sequential logic, start sensing dialects (Comb, Seq, HW..)

- 2. What is CIRCT ?

"CIRCT" stands for "Circuit Intermediate Representations (IR) Compilers and Tools". One might also interpret it as the recursively as "CIRCT IR Compiler and Tools". The T can be selectively expanded as Tool, Translator, Team, Technology, Target, Tree, Type, ... we're ok with the ambiguity.

- The tools of cirt
 - cirt-verilog
 - Verif (lec bmc)
 - Synthesis (cirt-opt cirt-synth)
- The dialects of cirt
 - Core (seq,comb hw)
 - What is upstream and donstream ?
 - It extends mlir (arith scf ..),
 - It can emits verilog, which is the upstream of fpga/asic flow

- 2. What is CIRCT ?

Transform Dialect

Transform Dialect

Owner: Max

Solar Eclipse

Solar Eclipse

Owner: Gabriel

Meeting Notes

2026-06-30

- Attendees
 - Kunwar, Gabriel, Markus, Theo, Maximilian, Matthias, Louis, Sasha, Jules
- Topics
 - Demo of tutorial environment
 - Do we use a dialect throughout tutorials?
 - Theo suggests re-using lislant from 2025 UnSchool
 - Use monorepo
 - Kunwar will move repository to GitHub org
 - And add people
 - Sasha: What should we do with sessions that we previously proposed?
 - Don't want it in the official schedule
 - Could fit it on the last day's presentations
 - Review lecture status on July 19th
 - Could be a good time to discuss extra presentations also
 - ACM needs a slot but have not yet confirmed whether they will be in person or online, and whether Wednesday afternoon slot works
 - Sasha and Jules are happy to help with course materials, ping on Discord
 - How many people will come?
 - Room can fit 200 people
 - Since there are multiple MLIR experts in the room should be able to scale to any number
 - Some of the people who have spaces are likely not to make it in the end, so will be slightly lower than number of people admitted
 - Recommendation for attendees is to first book travel etc and finally pay for school

2026-06-16

- Attendees
 - Matthias, Kunwar, Theo, Mathieu, Max Bartel, Sasha, Jules, Tobias, Gabriel
- Topics
 - First/Second track
 - Theo: should the second track be used for people who need more time for the first track?
 - Let's discuss this later, first making schedules/courses for first/second track then see how they'll interact
 - Sessions are 10-14 and 16-20
 - Wednesday is a half-day
 - Monday afternoon will have posters
 - Have morning session
 - Afternoon or half afternoon for posters?
 - If half then better to do last half of session, 6-8pm

- [Introduction session](#)
 - Lead: Kunwar, Support: Tobias
 - Will discuss offline how to organize
- School got accepted by ACM
 - Requirements:
 - They come and talk about ACM
 - Sessions will be recorded and hosted by ACM
- We already have glasses for the eclipse
- Some sessions that we initially proposed did not fit into the schedule
 - Tobias warns against scheduling too many things, proposed work already quite a lot
 - Better to first make sure that the core sessions are ready (within 2-3 weeks)
 - Feedback from 2025 Summer School was that introductory sessions first would be good
 - Move Mem2Reg to Thursday and Analysis to Wednesday?

2026-05-22

- Attendees
 - Matthias, Markus, Teho, Kunwar, Maximilian, Gabriel, Sasha, Mathieu
- Topics
 - Two rooms -> Two tracks - fast and slow
 - Schedule TBD
 - How do people know which track to attend?
 - Not clear right now
 - Maybe just do one track and do hacking in the second room?
 - Jules to teach PDL?
 - He has applied to join, which makes it easier to make him a teacher also, other teachers already confirmed
 - Schedule
 - See Schedule tab
 - Max shared [ML compiler exercises](#) made by an intern at Roofline
 - Topics:
 - Max
 - Scalable vectors
 - Transform dialect
 - Interfaces
 - Theo
 - TileIR
 - Can rent GPU to run on
 - mem2reg

- Matthias
 - Dataflow analysis
 - forwards/backwards
 - Bufferization
 - Rewrites
- Mathieu
 - Symbols
- Sasha
 - PDL
 - Eqsat in MLIR
 - Python bindings
 - Assembly dialects
- Kunwar
 - SMT dialect
 - Better for expert track
- Markus
 - MLIR C++ datastructures
 - How does SSA actually look like in C++?
 - Lowering to LLVM
 - Intro to LLVM from the perspective of an MLIR user
 - Tablegen deep dive
 - Tablegen the language & ODS
 - Maybe as part of dialect definition?
- Kunwar suggested adding more themes directly to the schedule tab for now offline
- Ordering:
 - Defining your own IR vs existing IRs
 - Sasha suggested using existing IRs first, then defining own, Kunwar suggested the other
 - How to design an IR probably the most valuable thing to teach
 - Can decide on the order later
- Overall Structure
 - Want the exercises to be uniform
 - At PPOP in Sydney distributing MLIR wheels built for four architectures worked consistently including on old laptops
 - Can include xDSL notebooks but Kunwar's objective is to have uniform exercises
 - Some notebooks don't show any Python APIs, these could still be useful to explain MLIR functionality, will need to see WRT schedule
- Meta
 - Will meet weekly for the first few weeks then potentially keep the slot but meet less regularly

- First priority is to decide on overall structure then can divide tasks

Two Track Discussion:

- Don't name them beginner/advanced, rather name them foundation/something-else
- A good idea is to have more explanations for "why some MLIR internals are built this way" in the non-foundational path
- Maybe have "expert boxes" in the lectures
- Have some facilitators in the room to help out, which also helps with getting people talking, etc.

Timeline

Monday	
10:00 - 11:45	Introduction (Kunwar, Tobias)
12:15 - 14:00	Compiler Introduction (Saday, Tobias)
14:00 - 16:00	Break
16:00 - 18:00	MLIR IR Introduction (Theo, Mathieu)
18:00 - 20:00	Poster Session

Tuesday	
10:00 - 12:00	ODS (Markus)
12:00 - 14:00	Building Transformations 1 (Mathias)
14:00 - 16:00	Break
16:00 - 18:00	Building Transformations 2 (Mathias)
18:00 - 20:00	Core Transformations (Markus, Max)

Wednesday	
10:00 - 12:00	Interfaces (Mathias, Kunwar)
12:00 - 14:00	Mem2Reg (Theo)
14:00 - 16:00	Break
16:00-17:00	SLOT AVAILABLE (ACM? - TBC)

17:00 - 20:00	Solar Eclipse
---------------	-------------------------------

Thursday	
10:00 - 12:00	Data Flow Analysis (Mathieu)
12:00 - 14:00	CUDA Tile IR (Theo)
14:00 - 16:00	Break
16:00 - 18:00	CIRCT (Louis)
18:00 - 20:00	Transform Dialect (Max)

Friday	
10:00 - 12:00	
12:00 - 14:00	Presentations
14:00 - 16:00	End